CAMPUS CAFE ORDERING SYSTEM

Java Program Code Explanation

Authors
Harold C. Kadusale
Anton Miguel M. Landisa

This document explains the purpose of each class, its important attributes and methods, and how the program works from creating customers and menu items to printing order receipts.

1. Overview of the Program

The Campus Cafe Ordering System is a Java program that represents a simple cafe ordering process. It allows the program to create customers, prepare different menu items, add items to an order, calculate prices and discounts, and print receipts. The program uses object-oriented programming concepts such as classes, objects, inheritance, abstraction, encapsulation, and method overloading.

Main features

• Creates student and non-student customers.

• Supports three menu item types: beverages, pastries, and meals.

• Calculates additional charges for drink sizes, warmed pastries, upsizing, and extra sides.

• Limits each order to three menu items.

• Applies a 10% student discount and an optional voucher.

• Awards loyalty points based on the amount paid.

2. Explanation of Each Java Class

CafeApp

This is the main class of the system. Its **main()** method starts the program and demonstrates how the other classes work together. It creates two customers, creates beverages, pastries, and meals, and also demonstrates the use of default constructors and setter methods. It tests price validation by attempting to set a negative price. Then it creates two orders, adds menu items, and prints the receipts. The first order also demonstrates the three-item order limit, while the second receipt demonstrates a voucher.

MenuItem

This is an **abstract parent class** for all menu items. It stores shared information: **name**, **basePrice**, and **itemCode**. The constructors set default or supplied values. The abstract method **computePrice()** requires each child class to provide its own price calculation. **getDescription()** returns the item name and code. Getter and setter methods control access to the fields. **setBasePrice()** rejects negative prices, helping protect the object's data.

Beverage

This class extends **MenuItem** and represents drinks. It adds a **size** field. Its **computePrice()** method adds ■25 for large drinks, ■15 for medium drinks, and no extra charge for small drinks. **getDescription()** adds the drink size to the item's description. **setSize()** accepts only S, M, or L.

Pastry

This class extends **MenuItem** and represents baked goods. It has a boolean field named **warmed**. Its **computePrice()** method adds ■10 when the pastry is warmed. Its description shows “(Warmed)” when that option is selected. The **isWarmed()** and **setWarmed()** methods read and update the warming option.

Meal

This class extends **MenuItem** and represents a full meal. It has two extra fields: **upsized** and **extraSides**. Its **computePrice()** method adds ■20 for an upsized meal and ■12 for each extra side. Its description displays whether the meal is regular or upsized and how many extra sides were selected. **setExtraSides()** prevents a negative number of sides.

Customer

This class stores customer information: **name**, whether the customer is a **student**, and the customer's **loyaltyPoints**. It has a no-argument constructor, a name-only constructor, and a constructor that accepts both name and student status. The two **addPoints()** methods demonstrate method overloading: one accepts an integer

number of points, while the other accepts an amount spent and awards one point for every ■50. Getter and setter methods provide controlled access to the fields.

Order

This class manages one customer order. It stores the customer, order ID, three menu-item references, an item counter, and a flag that tracks whether loyalty points were awarded. **addItem()** adds an item if a slot is available and rejects null items or a fourth item. **computeSubtotal()** adds the prices of the items. **computeTotal()** applies a 10% discount when the customer is a student. The overloaded **computeTotal(double voucher)** subtracts a voucher and prevents the total from going below zero. The overloaded **printReceipt()** methods print the receipt, with the no-argument version using no voucher. The receipt shows the items, subtotal, discount, voucher, final total, and loyalty points.

3. Important Object-Oriented Programming Concepts

Class and object

A class is a blueprint. An object is an instance created from that blueprint. For example, **Customer** is a class, while **customer1** and **customer2** are objects.

Encapsulation

Fields such as itemCode, size, warmed, and customer details are private or protected, and methods are used to access or update them. Validation in setters helps prevent invalid values.

Inheritance

Beverage, Pastry, and Meal use **extends MenuItem**. They inherit common menu-item information and behavior from MenuItem.

Abstraction

MenuItem is abstract and declares **computePrice()** without providing one shared calculation. Each child class supplies its own version.

Polymorphism

The Order class stores menu items using the parent type **MenuItem**. When computePrice() is called, Java runs the correct version for the actual object, such as Beverage, Pastry, or Meal.

Method overloading

The Customer class has two addPoints() methods with different parameter types. The Order class also has printReceipt() and computeTotal() methods with different parameter lists.

Method overriding

Beverage, Pastry, and Meal override computePrice() and getDescription() to provide behavior specific to each type.

4. Program Flow: Step by Step

1. The main() method creates Ana Reyes as a student customer and Ben Cruz as a non-student customer.

2. It creates an iced latte, ensaymada, and chicken adobo using parameterized constructors.

3. It creates a hot chocolate, cookie, and beef tapa using default constructors, then assigns values through setter methods.

4. It attempts to set the hot chocolate price to -5. The setter displays an error and keeps the previous valid price.

5. It creates order 1001 for Ana and adds the latte, ensaymada, and adobo.

6. It tries to add a fourth item, but the Order class rejects it because the order supports only three items.

7. It prints Ana's receipt. Her student discount is calculated as 10% of the subtotal.

8. It creates order 1002 for Ben and adds hot chocolate and beef tapa.

9. It prints Ben's receipt with a ■20 voucher. Since Ben is not a student, no student discount is applied.

10. When each receipt is printed for the first time, loyalty points are added based on the final total. The points are awarded only once per order.

5. Example Price Calculations

The following calculations show how the program applies its menu options. They are based on the values set in CafeApp.

Menu item	Calculation	Final price
Iced Latte (Large)	■95 + ■25	■120
Ensaymada (Warmed)	■45 + ■10	■55
Chicken Adobo (Upsized, 1 side)	■110 + ■20 + ■12	■142
Hot Chocolate (Medium)	■80 + ■15	■95
Beef Tapa (Regular, no sides)	■130 + ■0	■130

Note: The student discount is applied to the order subtotal. A voucher is then subtracted from the discounted total. The program does not allow the final total to become negative.

6. Important Notes About the Code

• Save each public Java class in its own file with the matching filename: CafeApp.java, MenuItem.java, Beverage.java, Pastry.java, Meal.java, Customer.java, and Order.java.

• The MenuItem class must be declared abstract because it contains the abstract method computePrice().

• The code uses the peso symbol in this explanation for readability. The Java program itself prints numeric prices without a currency symbol unless you add one to the receipt format.

• The current Order class has exactly three item slots. To support more items, it would need a collection such as an ArrayList.

• The pointsAwarded flag prevents the same order from awarding loyalty points more than once when its receipt is printed repeatedly.

7. Conclusion

The Campus Cafe Ordering System demonstrates how Java classes can work together to model a real-world ordering process. Inheritance reduces repeated code, abstraction defines a common menu-item structure, encapsulation protects data, and polymorphism allows each menu item to calculate its own price. Constructors, setters, method overloading, validation, discounts, vouchers, receipts, and loyalty points make the program a useful example of object-oriented programming.